

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Kernels

XCS229 Machine Learning

[Tengu Ma]

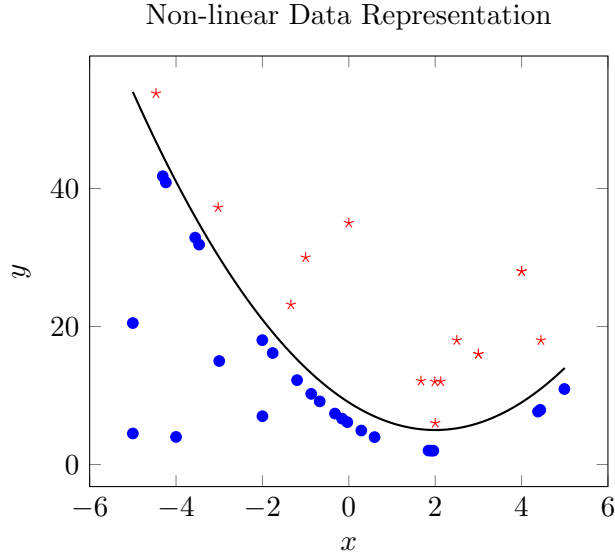
Contents

I	Introduction	1
II	Cubic Polynomial Fit	2
III	Implementing Linear Regression	3
IV	Challenges with High-Dimensional Feature Mapping	4
V	Summary	6

I. Introduction

In this lecture, we discuss the **kernel method**, a general technique for dealing with nonlinear relationships in data. We will explore how this method can be applied in supervised learning, particularly within discriminative algorithms, and will touch upon its application in unsupervised learning.

Lets consider a simple dataset in a regression setting where we have some data points denoted as (x, y) . The aim is to fit a model that appropriately represents the relationship in this dataset. Linear regression might not be appropriate if the data is inherently nonlinear.



II. Cubic Polynomial Fit

To deal with nonlinearity, we can use a cubic polynomial model:

$$h_{\theta}(x) = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0$$

where $x \in \mathbb{R}$.

Although $h_{\theta}(x)$ is nonlinear in x , it is linear in terms of the parameters θ . In linear regression, what really matters is that we are linear in our parameters θ because we are optimizing over the parameter space.

To turn our nonlinear model into a linear one, we can define a function $\phi(x)$ that maps x to a higher-dimensional vector:

$$\phi : \mathbb{R} \rightarrow \mathbb{R}^4$$

Specifically, we can define:

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

Then our model can be rewritten as:

$$\begin{aligned} h_{\theta}(x) &= \begin{bmatrix} \theta_0 & \theta_1 & \theta_2 & \theta_3 \end{bmatrix} \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \\ &= \theta^T \phi(x) \end{aligned}$$

Then $h_\theta(x)$ is linear in θ and $\phi(x)$.

Previously, we had a one-dimensional input x . Now, by using $\phi(x)$, we have transformed it into a four-dimensional vector. We can now redefine the dataset with new inputs as:

$$\xrightarrow{\text{new data set}} \left\{ \left(x^{(1)}, y^{(1)} \right), \left(x^{(2)}, y^{(2)} \right), \dots, \left(x^{(n)}, y^{(n)} \right) \right\} \\ \left\{ \left(\phi \left(x^{(1)} \right), y^{(1)} \right), \left(\phi \left(x^{(2)} \right), y^{(2)} \right), \dots, \left(\phi \left(x^{(n)} \right), y^{(n)} \right) \right\}$$

where $x^{(i)}, y^{(i)} \in \mathbb{R}$.

III. Implementing Linear Regression

We can implement linear regression on the new dataset

$$\left\{ \left(\phi \left(x^{(i)} \right), y^{(i)} \right) \right\}_{i=1}^n$$

using gradient descent. The update rule for gradient descent is given by:

$$\theta \leftarrow \theta + \alpha \cdot \nabla J(\theta)$$

Where $J(\theta) = \frac{1}{2} \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right)^2$. The gradient of the cost function can be expressed as:

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)})$$

We can rewrite it in a more iterative form:

$$\theta := \theta + \alpha \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)})$$

In essence, we are reiterating the linear regression procedures we learned previously. You just write down an algorithm for this new dataset and implement the linear regression algorithm on it.

Although we are repeating concepts from the first two weeks, it is essential to revisit them, as we will reuse these ideas to demonstrate something else, specifically, the **kernel trick**.

To perform linear regression on this new dataset, we will use gradient descent. We compare the label with the model, for which we have $\phi(x^{(i)})$.

Recall that with standard linear regression, the approach assumes that the model is:

$$h_\theta(x) = \theta^T x$$

Now, we replace $x^{(i)}$ with $\phi(x^{(i)})$.

Similarly, the corresponding stochastic gradient descent update rule is

$$\theta := \theta + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)})$$

In the standard case, both instances would refer directly to $x^{(i)}$. However, in our case, we are now utilizing the transformed feature version $\phi(x^{(i)})$.

Note that the dimensions of θ and $\phi(x)$ are different. While x is generally of dimension d , $\phi(x)$ maps $\mathbb{R}^d$ to some new dimension p ($\phi(x) : \mathbb{R}^d \rightarrow \mathbb{R}^p$).

Dimension of $\phi(x)$ is p .

This means θ must also be in the same space since it will be a linear combination of transformed inputs. Thus, we are updating in this p -dimensional space rather than the original x -dimensional space.

It's worth noting that this mapping is often referred to as a feature map. The function ϕ maps from $\mathbb{R}^d$ to $\mathbb{R}^p$, and in this context, $\phi(x)$ is often labeled as features.

While reviewing different papers on the subject, you might find that x is referenced as attributes to distinguish from features. The terms may vary slightly depending on the author, but the core understanding remains consistent.

In most cases, if a linear function is applied to a variable, it is considered a feature.

Note:

- While everything appears clear when substituting $x^{(i)}$ with $\phi(x^{(i)})$, we must recognize that there are still considerations to be made. In some scenarios, this approach works well, as seen in the homework example where you were requested to implement a similar process.
- However, the task was to utilize a different algorithm, specifically one that leveraged the inverse of the time-summed matrices in gradient calculations, showing that we can adopt multiple methods to tackle the same problem.

IV. Challenges with High-Dimensional Feature Mapping

As we apply our existing algorithm to the new dataset, we need to consider some computational efficiency issues that arise, particularly with high-dimensional feature maps.

For instance, let's consider $x \in \mathbb{R}^d$ and let $\phi(x)$ be the vector that captures all the monomials of x with degrees less than or equal to 3. The vector representation can be illustrated as follows:

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_1x_d \\ x_2x_1 \\ \vdots \\ x_d^2 \\ x_1^3 \\ x_1x_2^2 \\ \vdots \\ x_d^3 \end{bmatrix} \begin{array}{l} \textcolor{red}{\} 1 \\ \\ \textcolor{red}{\} d \\ \\ \textcolor{red}{\} d^2 \\ \\ \textcolor{red}{\} d^3 \end{array}$$

Every degree-3 polynomial can be expressed as $\theta^T \phi(x)$, where $\phi(x) \in \mathbb{R}^p$ and $p = 1 + d + d^2 + d^3$. For example, if $d = 10^3$, then:

$$p = 1 + 10^3 + (10^3)^2 + (10^3)^3 = 10^9$$

The update rule for gradient descent becomes:

$$\theta := \theta + \underbrace{\alpha \sum_{i=1}^n \left(y^{(i)} - \underbrace{\theta^T \phi(x^{(i)})}_{O(p)} \right) \phi(x^{(i)})}_{O(np) \text{ total operations}}$$

Due to the high dimensionality of $\phi(x)$, where p can be extremely large (e.g., 10^9), we encounter significant computational challenges. Specifically, each

iteration of gradient descent requires $O(p)$ operations for the inner product and an additional $O(np)$ operations for the entire update step.

This implies that if p is on the order of 10^9 and we have n samples, then the total number of required operations can become prohibitive in practice.

The objective of the next lecture is to introduce the **kernel trick**, which allows us to implement nonlinear models more efficiently without explicitly computing the high-dimensional feature mappings. The final algorithm will maintain the same functionality as the direct implementation but will be considerably faster and more efficient.

Machine learning is fundamentally about computational efficiency, and as we strive for better algorithms, understanding that computational aspects are crucial for applying our methods to large datasets is vital.

V. Summary

In this lecture, we explored the concept of the **kernel method** as a powerful technique for handling nonlinear relationships in data, particularly in the context of supervised learning. Here are the key takeaways:

- **Transformation of Inputs:** We illustrated how nonlinear functions can be handled by transforming inputs into higher-dimensional spaces using the feature mapping function $\phi(x)$.
- **Cubic Polynomial Fit:** We discussed the cubic polynomial model as an example, demonstrating that even nonlinear models can be expressed in terms of linear parameters when using an appropriate feature mapping.
- **Gradient Descent in High Dimensions:** We learned how to apply gradient descent to the new transformed dataset, highlighting the importance of retaining linearity in the parameter space while dealing with nonlinearity in the input space.
- **Challenges with High Dimensions:** The lecture addressed the computational challenges associated with high-dimensional feature mappings, illustrating how high dimensionality can lead to inefficiencies in algorithm performance.
- **Importance of Computational Efficiency:** A recurring theme was the significance of computational efficiency in machine learning, emphasizing that effective algorithms must balance both statistical accuracy and computational feasibility.